

Coroutines for Linux kernel

Marek Behún

LINUX
PLUMBERS
CONFERENCE
PRAGUE, CZECHIA / OCT. 5-7, 2026

```
/* The main state machine */
switch (sfp->sm_state) {
case SFP_S_DOWN:
        if (sfp->sm_mod_state != SFP_MOD_PRESENT ||
            sfp->sm_dev_state != SFP_DEV_UP)
                break;

        if (sfp->have_a2)
                sfp_soft_start_poll(sfp);

        sfp_module_tx_enable(sfp);

        sfp->sm_fault_retries = N_FAULT_INIT;

        sfp_sm_next(sfp, SFP_S_WAIT, sfp->module_t_wait);
        break;

case SFP_S_WAIT:
        if (event != SFP_E_TIMEOUT)
                break;

        if (sfp->state & SFP_F_TX_FAULT) {
                timeout = sfp->module_t_start_up;
                if (timeout > sfp->module_t_wait)
                        timeout -= sfp->module_t_wait;
                else
                        timeout = 1;

                sfp_sm_next(sfp, SFP_S_INIT, timeout);
        } else {
                goto init_done;
        }
        break;

case SFP_S_INIT:
        if (event == SFP_E_TIMEOUT && sfp->state & SFP_F_TX_FAULT) {
                sfp_sm_fault(sfp, SFP_S_INIT_TX_FAULT,
                             sfp->sm_fault_retries == N_FAULT_INIT);
        } else if (event == SFP_E_TIMEOUT || event == SFP_E_TX_CLEAR) {
init_done:
                ret = sfp_sm_add_mdio_bus(sfp);
                if (ret < 0) {
                        sfp_sm_next(sfp, SFP_S_FAIL, 0);
                        break;
                }
                sfp->sm_phy_retries = R_PHY_RETRY;
                goto phy_probe;
        }
        break;

case SFP_S_INIT_PHY:
        if (event != SFP_E_TIMEOUT)
                break;
phy_probe:
        ret = sfp_sm_probe_for_phy(sfp);
        if (ret == -ENODEV) {
                if (--sfp->sm_phy_retries) {
                        sfp_sm_next(sfp, SFP_S_INIT_PHY,
                                    sfp->phy_t_retry);
                        dev_dbg(sfp->dev,
                                "no PHY detected, %u tries left\n",
                                sfp->sm_phy_retries);
                        break;
                } else {
                        dev_info(sfp->dev, "no PHY detected\n");
                }
        } else if (ret) {
                sfp_sm_next(sfp, SFP_S_FAIL, 0);
```

```
/* The main state machine */
switch (sfp->sm_state) {
case SFP_S_DOWN:
        if (sfp->sm_mod_state != SFP_MOD_PRESENT ||
            sfp->sm_dev_state != SFP_DEV_UP)
                break;

        if (sfp->have_a2)
                sfp_soft_start_poll(sfp);

        sfp_module_tx_enable(sfp);

        sfp->sm_fault_retries = N_FAULT_INIT;

        sfp_sm_next(sfp, SFP_S_WAIT, sfp->module_t_wait);
        break;

case SFP_S_WAIT:
        if (event != SFP_E_TIMEOUT)
                break;

        if (sfp->state & SFP_F_TX_FAULT) {
                timeout = sfp->module_t_start_up;
                if (timeout > sfp->module_t_wait)
                        timeout -= sfp->module_t_wait;
                else
                        timeout = 1;

                sfp_sm_next(sfp, SFP_S_INIT, timeout);
        } else {
                goto init_done;
        }
        break;

case SFP_S_INIT:
        if (event == SFP_E_TIMEOUT && sfp->state & SFP_F_TX_FAULT) {
                sfp_sm_fault(sfp, SFP_S_INIT_TX_FAULT,
                             sfp->sm_fault_retries == N_FAULT_INIT);
        } else if (event == SFP_E_TIMEOUT || event == SFP_E_TX_CLEAR) {
init_done:
                ret = sfp_sm_add_mdio_bus(sfp);
                if (ret < 0) {
                        sfp_sm_next(sfp, SFP_S_FAIL, 0);
                        break;
                }
                sfp->sm_phy_retries = R_PHY_RETRY;
                goto phy_probe;
        }
        break;

case SFP_S_INIT_PHY:
        if (event != SFP_E_TIMEOUT)
                break;
phy_probe:
        ret = sfp_sm_probe_for_phy(sfp);
        if (ret == -ENODEV) {
                if (--sfp->sm_phy_retries) {
                        sfp_sm_next(sfp, SFP_S_INIT_PHY,
                                    sfp->phy_t_retry);
                        dev_dbg(sfp->dev,
                                "no PHY detected, %u tries left\n",
                                sfp->sm_phy_retries);
                        break;
                } else {
                        dev_info(sfp->dev, "no PHY detected\n");
                }
        } else if (ret) {
                sfp_sm_next(sfp, SFP_S_FAIL, 0);
```

```
/* The main state machine */
switch (sfp->sm_state) {
case SFP_S_DOWN:
        if (sfp->sm_mod_state != SFP_MOD_PRESENT ||
            sfp->sm_dev_state != SFP_DEV_UP)
                break;

        if (sfp->have_a2)
                sfp_soft_start_poll(sfp);

        sfp_module_tx_enable(sfp);

        sfp->sm_fault_retries = N_FAULT_INIT;

        sfp_sm_next(sfp, SFP_S_WAIT, sfp->module_t_wait);
        break;

case SFP_S_WAIT:
        if (event != SFP_E_TIMEOUT)
                break;

        if (sfp->state & SFP_F_TX_FAULT) {
                timeout = sfp->module_t_start_up;
                if (timeout > sfp->module_t_wait)
                        timeout -= sfp->module_t_wait;
                else
                        timeout = 1;

                sfp_sm_next(sfp, SFP_S_INIT, timeout);
        } else {
                goto init_done;
        }
        break;

case SFP_S_INIT:
        if (event == SFP_E_TIMEOUT && sfp->state & SFP_F_TX_FAULT) {
                sfp_sm_fault(sfp, SFP_S_INIT_TX_FAULT,
                             sfp->sm_fault_retries == N_FAULT_INIT);
        } else if (event == SFP_E_TIMEOUT || event == SFP_E_TX_CLEAR) {
init_done:
                ret = sfp_sm_add_mdio_bus(sfp);
                if (ret < 0) {
                        sfp_sm_next(sfp, SFP_S_FAIL, 0);
                        break;
                }
                sfp->sm_phy_retries = R_PHY_RETRY;
                goto phy_probe;
        }
        break;

case SFP_S_INIT_PHY:
        if (event != SFP_E_TIMEOUT)
                break;
phy_probe:
        ret = sfp_sm_probe_for_phy(sfp);
        if (ret == -ENODEV) {
                if (--sfp->sm_phy_retries) {
                        sfp_sm_next(sfp, SFP_S_INIT_PHY,
                                    sfp->phy_t_retry);
                        dev_dbg(sfp->dev,
                                "no PHY detected, %u tries left\n",
                                sfp->sm_phy_retries);
                        break;
                } else {
                        dev_info(sfp->dev, "no PHY detected\n");
                }
        } else if (ret) {
                sfp_sm_next(sfp, SFP_S_FAIL, 0);
```

```
/* The main state machine */
switch (sfp->sm_state) {
case SFP_S_DOWN:
        if (sfp->sm_mod_state != SFP_MOD_PRESENT ||
            sfp->sm_dev_state != SFP_DEV_UP)
                break;

        if (sfp->have_a2)
                sfp_soft_start_poll(sfp);

        sfp_module_tx_enable(sfp);

        sfp->sm_fault_retries = N_FAULT_INIT;

        sfp_sm_next(sfp, SFP_S_WAIT, sfp->module_t_wait);
        break;

case SFP_S_WAIT:
        if (event != SFP_E_TIMEOUT)
                break;

        if (sfp->state & SFP_F_TX_FAULT) {
                timeout = sfp->module_t_start_up;
                if (timeout > sfp->module_t_wait)
                        timeout -= sfp->module_t_wait;
                else
                        timeout = 1;

                sfp_sm_next(sfp, SFP_S_INIT, timeout);
        } else {
                goto init_done;
        }
        break;

case SFP_S_INIT:
        if (event == SFP_E_TIMEOUT && sfp->state & SFP_F_TX_FAULT) {
                sfp_sm_fault(sfp, SFP_S_INIT_TX_FAULT,
                             sfp->sm_fault_retries == N_FAULT_INIT);
        } else if (event == SFP_E_TIMEOUT || event == SFP_E_TX_CLEAR) {
init_done:
                ret = sfp_sm_add_mdio_bus(sfp);
                if (ret < 0) {
                        sfp_sm_next(sfp, SFP_S_FAIL, 0);
                        break;
                }
                sfp->sm_phy_retries = R_PHY_RETRY;
                goto phy_probe;
        }
        break;

case SFP_S_INIT_PHY:
        if (event != SFP_E_TIMEOUT)
                break;
phy_probe:
        ret = sfp_sm_probe_for_phy(sfp);
        if (ret == -ENODEV) {
                if (--sfp->sm_phy_retries) {
                        sfp_sm_next(sfp, SFP_S_INIT_PHY,
                                    sfp->phy_t_retry);
                        dev_dbg(sfp->dev,
                                "no PHY detected, %u tries left\n",
                                sfp->sm_phy_retries);
                        break;
                } else {
                        dev_info(sfp->dev, "no PHY detected\n");
                }
        } else if (ret) {
                sfp_sm_next(sfp, SFP_S_FAIL, 0);
```

```
/* The main state machine */
switch (sfp->sm_state) {
case SFP_S_DOWN:
        if (sfp->sm_mod_state != SFP_MOD_PRESENT ||
            sfp->sm_dev_state != SFP_DEV_UP)
                break;

        if (sfp->have_a2)
                sfp_soft_start_poll(sfp);

        sfp_module_tx_enable(sfp);

        sfp->sm_fault_retries = N_FAULT_INIT;

        sfp_sm_next(sfp, SFP_S_WAIT, sfp->module_t_wait);
        break;

case SFP_S_WAIT:
        if (event != SFP_E_TIMEOUT)
                break;

        if (sfp->state & SFP_F_TX_FAULT) {
                timeout = sfp->module_t_start_up;
                if (timeout > sfp->module_t_wait)
                        timeout -= sfp->module_t_wait;
                else
                        timeout = 1;

                sfp_sm_next(sfp, SFP_S_INIT, timeout);
        } else {
                goto init_done;
        }
        break;
```

```
case SFP_S_INIT:
        if (event == SFP_E_TIMEOUT && sfp->state & SFP_F_TX_FAULT) {
                sfp_sm_fault(sfp, SFP_S_INIT_TX_FAULT,
                             sfp->sm_fault_retries == N_FAULT_INIT);
        } else if (event == SFP_E_TIMEOUT || event == SFP_E_TX_CLEAR) {
init_done:
                ret = sfp_sm_add_mdio_bus(sfp);
                if (ret < 0) {
                        sfp_sm_next(sfp, SFP_S_FAIL, 0);
                        break;
                }
                sfp->sm_phy_retries = R_PHY_RETRY;
                goto phy_probe;
        }
        break;

case SFP_S_INIT_PHY:
        if (event != SFP_E_TIMEOUT)
                break;
phy_probe:
        ret = sfp_sm_probe_for_phy(sfp);
        if (ret == -ENODEV) {
                if (--sfp->sm_phy_retries) {
                        sfp_sm_next(sfp, SFP_S_INIT_PHY,
                                    sfp->phy_t_retry);
                        dev_dbg(sfp->dev,
                                "no PHY detected, %u tries left\n",
                                sfp->sm_phy_retries);
                        break;
                } else {
                        dev_info(sfp->dev, "no PHY detected\n");
                }
        } else if (ret) {
                sfp_sm_next(sfp, SFP_S_FAIL, 0);
```

```
/* The main state machine */
switch (sfp->sm_state) {
case SFP_S_DOWN:
        if (sfp->sm_mod_state != SFP_MOD_PRESENT ||
            sfp->sm_dev_state != SFP_DEV_UP)
                break;

        if (sfp->have_a2)
                sfp_soft_start_poll(sfp);

        sfp_module_tx_enable(sfp);

        sfp->sm_fault_retries = N_FAULT_INIT;

        sfp_sm_next(sfp, SFP_S_WAIT, sfp->module_t_wait);
        break;

case SFP_S_WAIT:
        if (event != SFP_E_TIMEOUT)
                break;

        if (sfp->state & SFP_F_TX_FAULT) {
                timeout = sfp->module_t_start_up;
                if (timeout > sfp->module_t_wait)
                        timeout -= sfp->module_t_wait;
                else
                        timeout = 1;

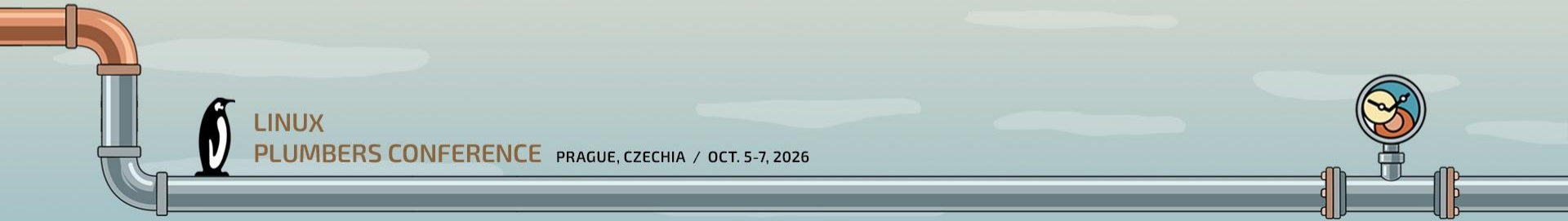
                sfp_sm_next(sfp, SFP_S_INIT, timeout);
        } else {
                goto init_done;
        }
        break;

case SFP_S_INIT:
        if (event == SFP_E_TIMEOUT && sfp->state & SFP_F_TX_FAULT) {
                sfp_sm_fault(sfp, SFP_S_INIT_TX_FAULT,
                             sfp->sm_fault_retries == N_FAULT_INIT);
        } else if (event == SFP_E_TIMEOUT || event == SFP_E_TX_CLEAR) {
init_done:
                ret = sfp_sm_add_mdio_bus(sfp);
                if (ret < 0) {
                        sfp_sm_next(sfp, SFP_S_FAIL, 0);
                        break;
                }
                sfp->sm_phy_retries = R_PHY_RETRY;
                goto phy_probe;
        }
        break;

case SFP_S_INIT_PHY:
        if (event != SFP_E_TIMEOUT)
                break;
phy_probe:
        ret = sfp_sm_probe_for_phy(sfp);
        if (ret == -ENODEV) {
                if (--sfp->sm_phy_retries) {
                        sfp_sm_next(sfp, SFP_S_INIT_PHY,
                                    sfp->phy_t_retry);
                        dev_dbg(sfp->dev,
                                "no PHY detected, %u tries left\n",
                                sfp->sm_phy_retries);
                        break;
                } else {
                        dev_info(sfp->dev, "no PHY detected\n");
                }
        } else if (ret) {
                sfp_sm_next(sfp, SFP_S_FAIL, 0);
```

Motivational example – sfp.c main state machine

- `switch()` statement
- 10 enumerated cases
  - `SFP_S_DOWN, SFP_S_WAIT, SFP_S_INIT, ...`
- in each call
  - one case is executed (* unless `goto`)
  - case for next call is configured
  - `break` and `return`
  - basically suspends execution at break and then
- case variable only changed within this function

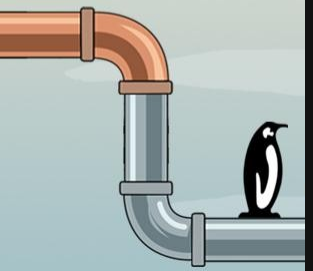


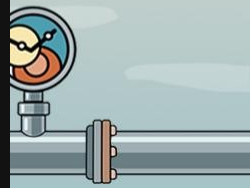
Closer look at suspend / resume point

```
void sfp_sm_main(struct sfp *sfp, unsigned int event) {
        switch (sfp->sm_state) {
        case SFP_S_DOWN:
                sfp_sm_next(sfp, SFP_S_WAIT, 50);
                break;

        case SFP_S_WAIT:
                if (sfp->state & SFP_F_TX_FAULT)
                        sfp_sm_next(sfp, SFP_S_INIT, 50);
                else
                        goto init_done;
                break;

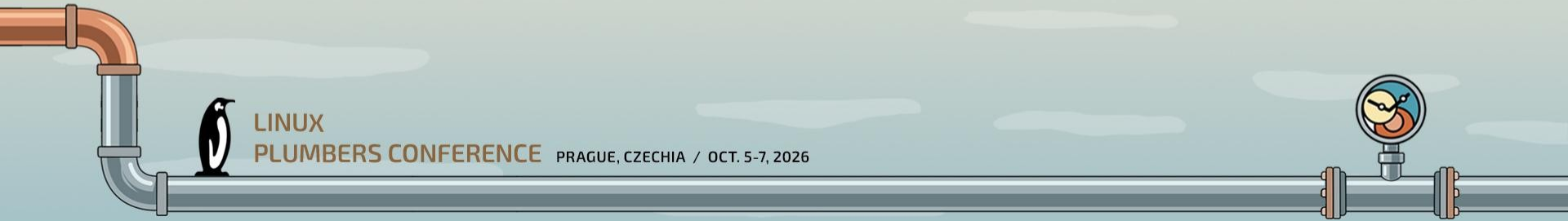
        case SFP_S_INIT:
        init_done:
                /* ... */
        }
}
```





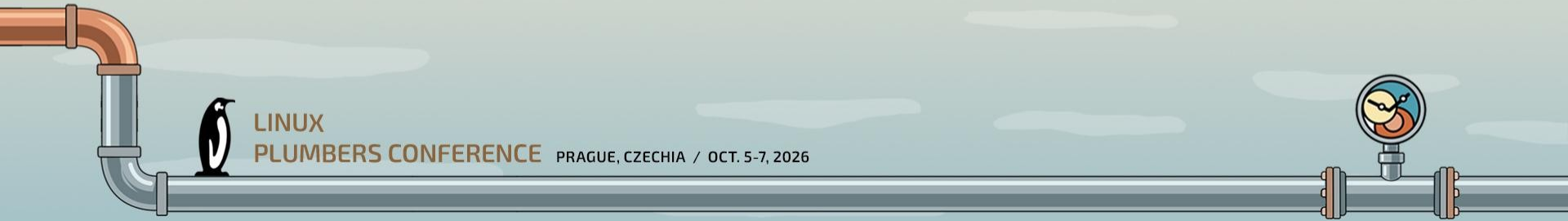
Understanding it is non-trivial

- when trying to understand
  - track disjointed code paths
  - or draw state diagrams
- contains code that translates to standard loops
  - `for (int i = 0; i < R_PHY_RETRY; ++i)`
  - loop logic hidden inside goto and goto-like jumps



Question

Can we have both asynchronous execution and tractable code?



Step 1 – hide switch() and cases behind macros

- `#define await(_s) \`
  `break; case _s:`

- `await_begin()` hides `switch()`
- `await_end()` hides the default, unreachable case

- Cons
  - state name repetition
  - async operation is called before await()

```
#define await_begin(_cur_state, _initial_state) \
        switch (_cur_state) {                   \
                case _initial_state:
#define await_end() \
        default: unreachable(); }

#define await(_s) \
        break; case _s:

void sfp_sm_main(struct sfp *sfp, unsigned int event) {
        await_begin(sfp->sm_state, SFP_S_DOWN) {
                sfp_sm_next(sfp, SFP_S_WAIT, 50);
                await(SFP_S_WAIT);

                if (sfp->state & SFP_F_TX_FAULT) {
                        sfp_sm_next(sfp, SFP_S_INIT, 50);
                        await(SFP_S_INIT);
                }

                /* ... */
        } await_end();
}
```

Step 2 – put async expression into await()

- `#define await(_expr, _s) \`
  `_expr; *_p = _s; \`
  `break; case _s:`

- `await()` now takes the asynchronous expression as an argument and sets new state

- Cons
  - `await_begin()` requires initial state
  - `await_end()`

```
#define await_begin(_cur_state, _initial_state) \
        typeof(_cur_state) *_p = &_cur_state;   \
        switch (_cur_state) {                   \
                case _initial_state:
#define await_end() \
        default: unreachable(); }

#define await(_expr, _s) \
        _expr; *_p = _s; break; case _s:

void sfp_sm_main(struct sfp *sfp, unsigned int event) {
        await_begin(sfp->sm_state, SFP_S_DOWN) {
                await(sfp_wait(sfp, 50), SFP_S_WAIT);

                if (sfp->state & SFP_F_TX_FAULT)
                        await(sfp_wait(sfp, 50), SFP_S_I

                /* ... */
        } await_end();
}
```

Step 3 – get rid of explicit state name

- `#define await(_expr)  \`
  `_expr;  \`
  `_state = __LINE__;  \`
  `break;  \`
  `case __LINE__:`

- `await()` does not need state name, uses `__LINE__` macro

- Cons
  - still `await_end()`
  - special scope for `await()`

```
#define await_begin(_state)             \
        typeof(_state) *_p = &_state;   \
        switch (_state) { case 0:
#define await_end() \
        default: unreachable(); }

#define await(_expr)            \
        _expr; *_p = __LINE__;  \
        break; case __LINE__:

void sfp_sm_main(struct sfp *sfp, unsigned int event)
        await_begin(sfp->sm_state) {
                await(sfp_wait(sfp, 50));

                if (sfp->state & SFP_F_TX_FAULT)
                        await(sfp_wait(sfp, 50));

                /* ... */
        } await_end();
}
```

Digression: Computed Gotos

- `goto` labels can be stored into a pointer
  - `void *ptr = &&label;`
  - `goto *ptr;`

- not in standard C, but part of GNU C
- supported by both GCC and Clang

- we can use this instead of `switch()`

```
void normal_goto_fn() {
        goto label;

label:
        return;
}

void computed_goto_fn() {
        void *ptr;

        ptr = &&label;

        goto *ptr;

label:
        return;
}
```



Step 4 – resume with computed goto

- ```
  #define await(_expr)  \
     _expr;              \
     jump_ptr = &&label; \
     return;              \
     label:
  ```

- `await()` saves label where it should continue at next resume into a pointer and suspends the function

- Cons
  - static pointer variable

```
#define _await(_lbl, _expr)     \
        (_expr);                \
        *_p = &&_lbl;           \
        return;                 \
        _lbl:
#define await(_expr)            \
        _await(__UNIQUE_ID(_lbl), _expr)

#define await_begin(_ptr)       \
        void **_p = &_ptr;      \
        if (_ptr)               \
                goto *_ptr;

void sfp_sm_main(struct sfp *sfp, unsigned int event) {
        await_begin(sfp->sm_ptr);

        await(sfp_wait(sfp, 50));

        if (sfp->state & SFP_F_TX_FAULT)
                await(sfp_wait(sfp, 50));

        /* ... */
}
```
```

Step 5 – create coroutine object

- `struct coroutine` contains the pointer
- `co_begin()` jumps with goto if the pointer is non-NULL
- `co_await()` saves the pointer and returns from the function (suspends the coroutine)

- Cons
  - static pointer variable

```
struct coroutine {
        void *ptr;
};

#define co_begin()                                      \
        do {                                            \
                if (coro->ptr)                          \
                        goto *coro->ptr;                \
        } while (0)

#define _co_await(_lbl, _expr)                  \
        do {                                    \
                (_expr);                        \
                coro->ptr = &&_lbl;             \
                return;                         \
                _lbl:                           \
        } while (0)

#define co_await(_expr) \
        _co_await(__UNIQUE_ID(_lbl), _expr)
```

Step 5 – create coroutine object

```
void sfp_main_coro(struct coroutine *coro, unsigned int event)
{
        struct sfp *sfp = coro_to_sfp(coro);

        co_begin();

        co_await(sfp_wait(sfp, 50));

        if (sfp->state & SFP_F_TX_FAULT)
                co_await(sfp_wait(sfp, 50));

        /* ... */
}
```





Step 6 – local variables

- we can’t use local variables across suspend points
  - `co_await()` returns from the function
  - stack variables are discarded

```
void sfp_main_coro(struct coroutine *co
{
        int var;

        co_begin();

        var = 5;

        co_await(coro_wait(coro, 50));

E>      var = var + 1;
  Variable 'var' is uninitialized when u

        /* ... */
}
```

Step 6 – local variables

- declare local variables inside an anonymous structure and create a pointer to that structure
- on coroutine entry `co_begin(v)` allocates space for the variables on heap and stores the pointer into `v` and `coro->vars`
- on resume `co_begin(v)` simply sets `v = coro->vars`
- Cons
  - we lose “unused variable” warnings

```
void sfp_main_coro(struct coroutine *coro
{
        struct {
                int var;
        } *v;

        co_begin(v);

        v->var = 5;

        co_await(coro_wait(coro, 50));

        v->var = v->var + 1;

        /* ... */
}
```



Step 6 – local variables

```
struct coroutine {
        void *ptr;
        void *vars;
};

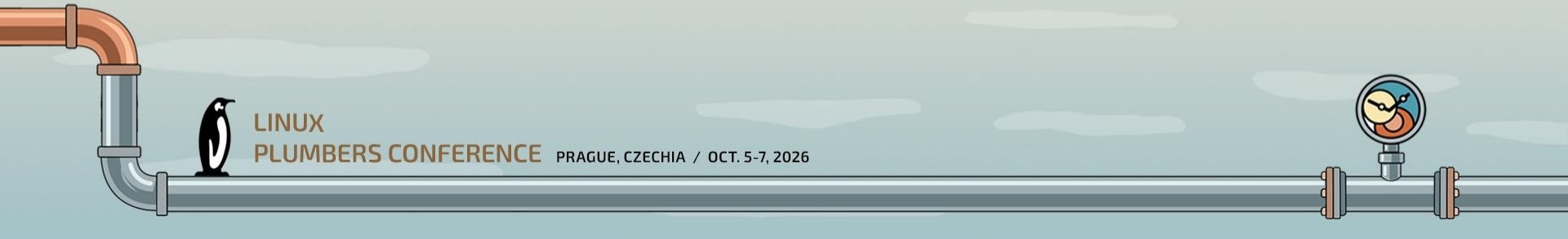
#define co_begin(v)                                 \
        do {                                        \
                if (coro->ptr) {                    \
                        v = coro->vars;             \
                        goto *coro->ptr;            \
                } else {                            \
                        v = malloc(sizeof(*v));     \
                        coro->vars = v;             \
                }                                   \
        } while (0)
```





Step 7 – awaiting values

- `co_await()` is now a `do … while (0)` statement
- it is not an expression, we cannot do
  - `res = co_await(…);`
  - `if (co_await(...))`
- let’s try convering it to statement expression (GNU C), which can return a value



Step 7 – awaiting values

```
#define _co_await(_lbl, _expr)          \
        ({                              \
                (_expr);                \
                coro->ptr = &&_lbl;     \
                return;                 \
                _lbl:                   \
                coro->await_result;     \
        })

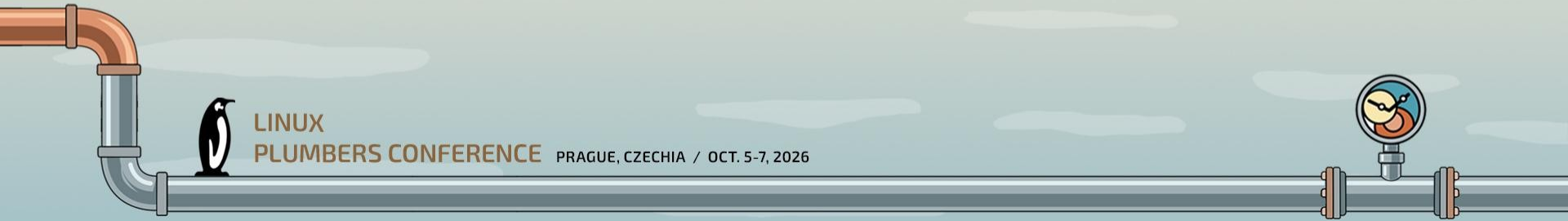
void sfp_main_coro(struct coroutine *coro)
{
E>      int event;
 Variable 'event' set but not used

E>      co_begin();
 Cannot jump from this indirect goto statement to one of its possible targets

        event = co_await(sfp_wait_for_event());
}
```

Step 7 – awaiting values

- both GCC and Clang forbid jumping into statement expressions
  - no `goto`, nor computed `goto`, nor `switch case`
- undefined behavior for multiple reasons
- GCC actually does not notice if computed `goto` is used and it works
  - but still cannot be used, since undefined behavior



Step 7 – awaiting values

- Solution
  - `co_await_into(res, expr)`
  - keep in `do … while (0)` block
  - tell in macro argument where to store awaited result
- Advantage
  - we can enforce return type of the expression to be compatible with result

```
#define _co_await_into(_lbl, _res, _expr)
        do {
                (_expr);
                coro->ptr = &&_lbl;
                return;
                _lbl:
                (_res) = coro->await_result;
        } while (0)

void sfp_main_coro(struct coroutine *coro)
{
        int event;

        co_begin();

        co_await_into(event, sfp_wait_for_event(
}
```

Step 7 – awaiting values – type safety

- if the awaited value is of type T, the expression put into `co_await_into()` must be of type `co_awaitable(T)`
- `co_awaitable(T)`
  - pointer to structure
  - the only member is function pointer `result_type()` returning type T
  - it is never called, only its type is used to check type compatibility

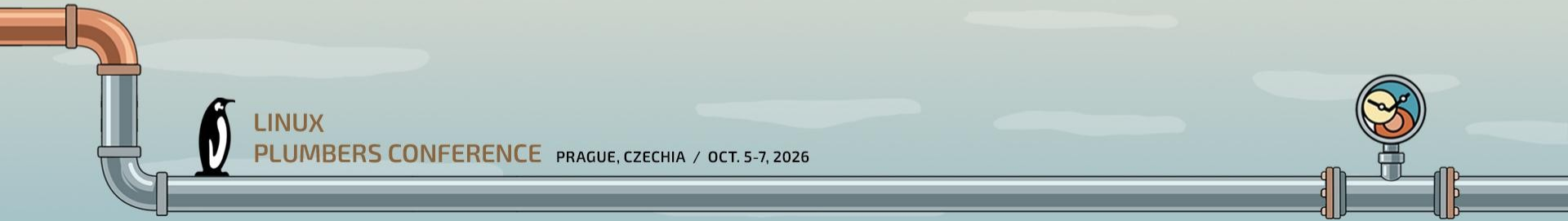
```
#define co_awaitable(_type)                 \
    const struct {                          \
        _type (*result_type)(void);         \
    } *

#define _co_await_check(var, expr)          \
    if (0) {                                \
        typeof((expr)->result_type()) \
            r();                            \
        (var) = r();                        \
    }

co_awaitable(int) sfp_wait_for_event();
```

Step 8 – allow co_await() not to suspend

- if awaited event is already ready when `co_await(expr)` is called, it should not suspend the coroutine
- how to make `co_await()` differentiate whether to suspend or not?
- `expr` is of type `co_awaitable(T)`, but it's value is not yet used, only it's type
- we can use the value to differentiate
  - `await_suspend = NULL` for suspend
  - `await_resume =` something non-`NULL` for resume



Step 8 – allow co_await() not to suspend

```
static const void *await_suspend = NULL;
static const void *await_ready = (void *)0x1000;

co_awaitable(int) sfp_wait_for_event()
{
        if (sfp->event_ready) {
                sfp->coro.await_result = sfp->event;
                return await_ready;
        }

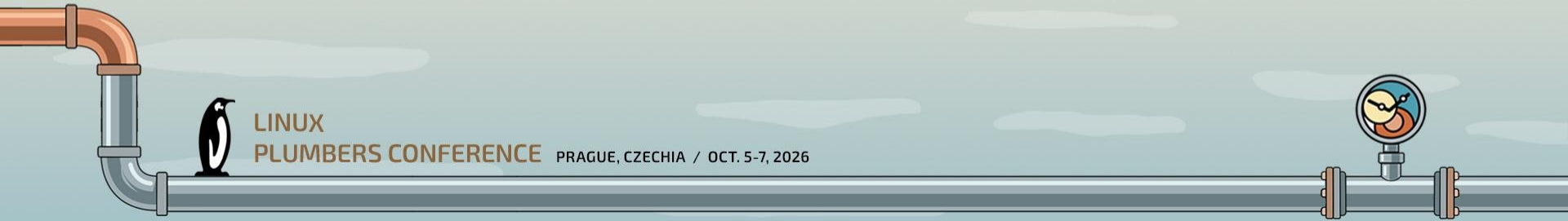
        return await_suspend;
}
```





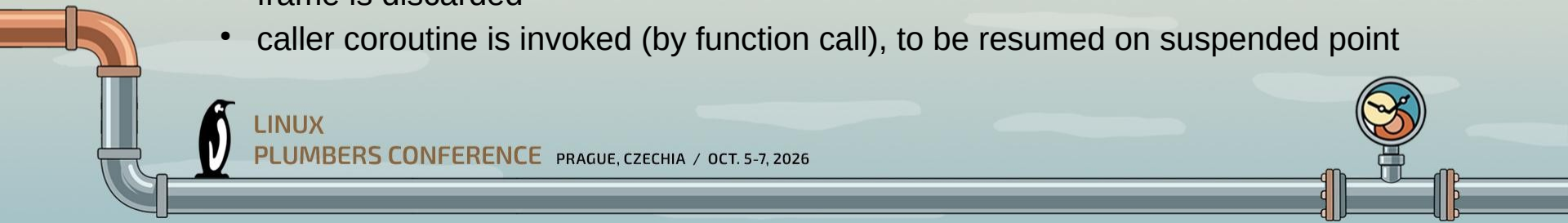
Step 9 – nested coroutines

- If we call `co_await(wait_for_event())` and event is not ready, the `wait_for_event()` function has to
  - arrange for the coroutine to be resumed later
  - return immediately
- `wait_for_event()` itself cannot be a coroutine now
  - it cannot suspend it's own execution with nested `co_await()`
  - it cannot co-return a value to the caller
- how to make this possible?



Step 9 – nested coroutines

- a stack of nested frames is needed in the coroutine object
- each frame contains
  - address of it’s function
  - where inside the function to jump on resume
  - local variables
  - size of the frame
- when invoking a coroutine
  - frame is created and filled (buffer grown with `realloc()`)
- on `co_return()`
  - returned result is stored
  - frame is discarded
  - caller coroutine is invoked (by function call), to be resumed on suspended point



Step 9 – nested coroutines

- a “runtime library” is needed for frame pushing / popping
- small enough to be header-only

```
struct coroutine {
        void *stack;
        size_t size;
        size_t frame;
};

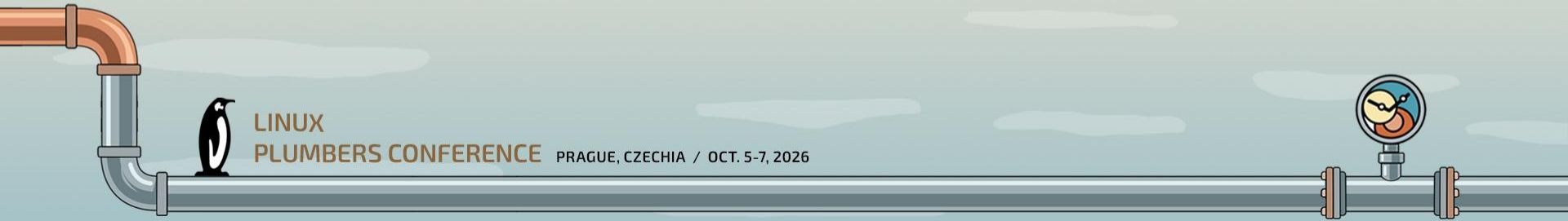
void *coro_alloc_frame(struct coroutine *coro);
void *coro_frame_vars(struct coroutine *coro);
void coro_pop_frame(struct coroutine *coro);
void *coro_jump_ptr(struct coroutine *coro);

#define co_begin(_vars)                                 \
        do {                                            \
                if (coro_jump_ptr(coro)) {              \
                        _vars = coro_frame_vars(coro);  \
                        goto *coro_jump_ptr(coro);      \
                } else {                                \
                        _vars = coro_alloc_frame(coro); \
                }                                       \
        } while (0)

#define co_return(_res) ...
#define co_return_void() ...
```

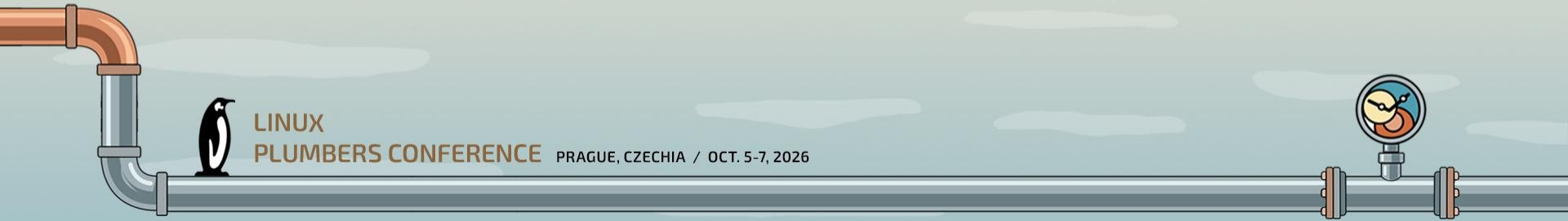
Not covered here

- argument passing



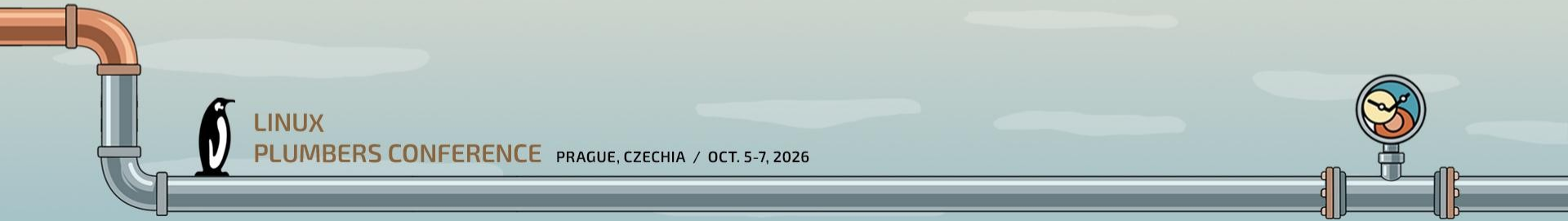
Disadvantages

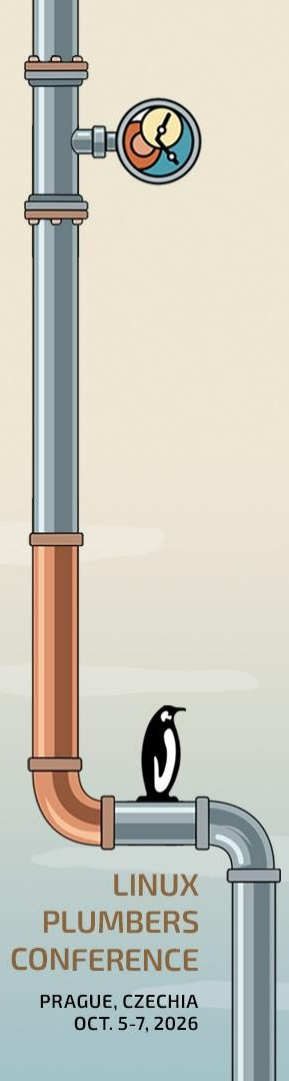
- local variables have to be accessed via `v->`
  - also no warnings for unused local variables
- we cannot use variable cleanup functions across suspend points
  - no guard() or scoped_guard()
  - no for_each_*_scoped()
  - but clang at least fails to compile if we try
- standard stack-trace does not reflect coroutine backtrace
- people who do not understand how it works may be burned



Advantages

- more tractable code for intended cases (sfp.c)
- API similar to C++20 coroutines
- no inline assembly (at least for now)
  - no arch specific code
- compiler can tail-call optimize if deeply nested coroutine suspends
- can be used in interrupt context





Questions?